

DynamoDB API Compatibility Test Scenarios

Table Lifecycle Operations

CreateTable

- **Basic create (PK only):** Create a table with a simple primary key.
Input: JSON with `TableName`, `KeySchema` (HASH key), and matching `AttributeDefinitions` (e.g. `{ "TableName": "T1", "KeySchema": [{ "AttributeName": "id", "KeyType": "HASH" }], "AttributeDefinitions": [{ "AttributeName": "id", "AttributeType": "S" }], "BillingMode": "PAY_PER_REQUEST" }`).
Expected: HTTP 200. Response JSON contains `TableDescription` with `TableStatus`: "CREATING" (the table is created asynchronously) ¹. (Note: `CreationDateTime`, `TableArn`, etc. are returned but may vary.) Eventually, `DescribeTable` should show `TableStatus`: "ACTIVE".
- **Create with sort key:** Same as above but include a composite primary key (HASH and RANGE).
Input: Add a `KeySchema` entry for sort key and its definition in `AttributeDefinitions`.
Expected: HTTP 200, `TableStatus`: "CREATING" ¹. Both keys appear in the returned `KeySchema`.
- **Missing TableName:** Omit `TableName` in the request.
Expected: HTTP 400 with a `ValidationException`. DynamoDB reports a missing required parameter or field ². The error message indicates the missing `TableName`.
- **Missing/invalid KeySchema or AttributeDefinitions:** For example, specify a key schema referring to an undefined attribute.
Input: E.g. `KeySchema` uses attribute "pk" but `AttributeDefinitions` omits "pk".
Expected: HTTP 400 `ValidationException`. The message notes the mismatch, e.g. "Some index key attributes are not defined in AttributeDefinitions..." ³ or "Number of attributes in KeySchema does not exactly match number of attributes defined in AttributeDefinitions" ⁴.
- **Duplicate table name:** Call `CreateTable` with the same name twice.
Expected: HTTP 400 `ResourceInUseException` with message like "The resource being changed is in use" ⁵. (DynamoDB does not allow duplicate names in a region.)
- **Invalid table name or parameters:** Use an invalid table name (e.g. empty string) or invalid throughput values.
Expected: HTTP 400 `ValidationException` for invalid parameter value ².

ListTables

- **No tables:** When no tables exist.
Input: `{ "TableName": null }` or empty request.
Expected: HTTP 200 with JSON `{ "TableNames": [] }` (no `LastEvaluatedTableName`) ⁶. An empty list signals no tables.
- **Multiple tables (pagination):** Create multiple tables (e.g. 3). Use `Limit` to paginate.
Input: `{ "Limit": 2 }`.

Expected: HTTP 200, JSON including up to 2 names in `TableNames`, plus a `LastEvaluatedTableName` if more remain ⁷. On a second call using `ExclusiveStartTableName` equal to the previous `LastEvaluatedTableName`, get the next page. Example: `{"TableNames": ["T1", "T2"], "LastEvaluatedTableName": "T2"}` then `{"TableNames": ["T3"]}`.

- **Invalid parameters:** Use `Limit` = 0 or >100.

Expected: HTTP 400 `ValidationException` (limit out of range) ². Similarly, a non-string `ExclusiveStartTableName` yields `ValidationException`.

DescribeTable

- **Existing table:** Describe a recently created table (after it becomes active).

Input: `{ "TableName": "T1" }`.

Expected: HTTP 200. Response JSON has a `Table` object with details: `TableStatus` ("CREATING" immediately after creation or "ACTIVE" later), `KeySchema`, `AttributeDefinitions`, item count (likely 0), etc. Fields like `CreationDateTime` are returned (and may be normalized in tests).

Example snippet: `"TableStatus": "ACTIVE"` ⁸ ⁹.

- **Non-existent table:** Call `DescribeTable` on a name that doesn't exist.

Expected: HTTP 400 `ResourceNotFoundException` with message "The resource being requested does not exist." ¹⁰.

- **Immediate consistency caveat:** (Optional) If `DescribeTable` is called immediately after `CreateTable`, DynamoDB might still return `ResourceNotFound` due to eventual consistency ¹¹. Tests should allow for brief retry or normalization.

DeleteTable

- **Existing table:** Delete a table that is `ACTIVE`.

Input: `{ "TableName": "T1" }`.

Expected: HTTP 200. The response JSON includes `TableDescription` with `"TableStatus": "DELETING"` (as deletion proceeds) ¹². Example:

```
{
  "TableDescription": {
    "TableName": "T1",
    "TableStatus": "DELETING",
    ...
  }
}
```

- **Deleting in CREATING/UPDATING:** Delete a table that is still `CREATING` or `UPDATING`.

Expected: HTTP 400 `ResourceInUseException` (cannot delete while in use) ¹³.

- **Already deleting (idempotency):** Delete the same table again immediately.

Expected: HTTP 200 with no error (DynamoDB treats re-deletion as no-op) ¹³. The returned `TableStatus` remains "DELETING".

- **Non-existent table:** Delete a table name that does not exist.

Expected: HTTP 400 `ResourceNotFoundException` ¹³.

- **Missing TableName:** Omit `TableName`.
Expected: HTTP 400 `ValidationException` (missing required parameter).

Item (CRUD) Operations

PutItem

- **Insert new item (no ReturnValues):** Add an item with only the primary key attributes (and any others).
Input: e.g. `{ "TableName": "T1", "Item": { "id": { "S": "123" } } }`.
Expected: HTTP 200, empty JSON (`{}`) since no ReturnValues was requested. (DynamoDB returns an empty body on success ¹⁴.) The item is inserted.
- **Replace existing item:** Put an item with the same key as an existing item (thus replacing it).
Input: Item with same `id` but different attributes.
Expected: HTTP 200. If `ReturnValues: "ALL_OLD"` is specified, response's `Attributes` contains the old item (pre-replacement) ¹⁴. If no `ReturnValues`, response is `{}`. Both calls succeed (PutItem is idempotent on identical payloads).
- **Idempotency of PutItem:** Call `PutItem` twice with identical data.
Expected: Both calls HTTP 200. The second call simply overwrites with the same values; no error is returned (PutItem is effectively idempotent in this case).
- **Missing TableName or Item:** Omit `TableName`, or omit the `Item` map.
Expected: HTTP 400 `ValidationException` (DynamoDB requires both) ².
- **Missing required key attribute:** Provide an `Item` that lacks the partition key (or sort key, if table has one).
Expected: HTTP 400 `ValidationException`. For example, if `id` is the key, message "Missing the key id in the item" ¹⁵.
- **Key name case sensitivity:** DynamoDB is case-sensitive for attribute names. E.g. table's key is `"Id"`, but request uses `"id"`. This is equivalent to missing the key.
Expected: HTTP 400 `ValidationException` (missing key with correct case) ¹⁵.
- **Type mismatch for key:** If the item's key attribute type does not match the table schema (e.g. key defined as Number but provided as String), DynamoDB rejects it.
Expected: HTTP 400 `ValidationException`. The message indicates a type mismatch. (AWS requires key types to match ¹⁶.)
- **Empty string or null key value:** If a key attribute (partition or sort) is given as an empty string or null, it is invalid. For example, `"id": { "S": "" }`.
Expected: HTTP 400 `ValidationException` – DynamoDB forbids empty string keys ¹⁷.
- **Invalid attribute value type:** Providing an unsupported type for an attribute (e.g. a non-JSON value, or a Boolean on a key when only S/N allowed) yields an error.
Expected: HTTP 400 `ValidationException` (invalid type) ¹⁶.
- **Reserved word in attribute name:** (Advanced) Using a DynamoDB reserved word in an expression without `ExpressionAttributeNames` causes a `ValidationException`. E.g. putting a condition on an attribute named "Name".
Expected: HTTP 400 `ValidationException` with message about reserved word. Tests should use `ExpressionAttributeNames` or avoid reserved words.

GetItem

- **Existing item:** Retrieve an item by its full key.
Input: `{ "TableName": "T1", "Key": {"id":{"S":"123"}} }`.
Expected: HTTP 200. JSON with `"Item": { ... }` containing the item's attributes. (E.g. `"Item": {"id":{"S":"123"}, "foo":{"S":"bar"}}`). If `ProjectionExpression` is used, only selected attributes appear.
- **Non-existent item:** Key is valid format but no such item exists.
Expected: HTTP 200 with no `Item` element (empty result) ¹⁸. DynamoDB does not error; the response JSON simply lacks an `Item` field (or returns an empty object).
- **Missing TableName or Key:** Omit one.
Expected: HTTP 400 `ValidationException` (missing required parameters) ².
- **Partial key on composite table:** Table has sort key but request provides only partition key.
Expected: HTTP 400 `ValidationException` ("ValidationException: Number of key attributes does not match schema" or similar).
- **Key type mismatch:** Provide wrong type for key (e.g. N vs S).
Expected: HTTP 400 `ValidationException` (type mismatch) ¹⁶.
- **Case mismatch in key name:** Using wrong-case attribute name in Key is like missing the key.
Expected: HTTP 400 `ValidationException` (missing key) ¹⁵.
- **ConsistentRead flag:** If `ConsistentRead:true` is set, GetItem uses strong consistency. (No change in response format.) Not an error case but ensure the flag is supported by both endpoints.

UpdateItem

- **Update existing item (add/modify attributes):**
Input: Include full `Key`, and either `AttributeUpdates` (legacy) or `UpdateExpression`. E.g. add a new attribute or change a number.
Expected: HTTP 200. With no `ReturnValues`, response is `{}`. The item is updated. If `ReturnValues:"ALL_NEW"` is set, the response's `Attributes` contains the updated item state ¹⁹.
- **Upsert (item does not exist):** If table has no item with given key and no condition is used, `UpdateItem` will create a new item with the given key and applied updates.
Input: `Key` for non-existent item, `UpdateExpression: "SET foo = :v"`.
Expected: HTTP 200. Item is created. With `ReturnValues:"ALL_NEW"`, response contains the new item.
- **Missing TableName or Key:**
Expected: HTTP 400 `ValidationException` ².
- **Missing sort key on composite table:** Only HASH given.
Expected: HTTP 400 `ValidationException` (key schema mismatch).
- **Key type mismatch:** Wrong type for key attribute.
Expected: HTTP 400 `ValidationException` (type mismatch) ¹⁶.
- **Empty string or null key:** Invalid.
Expected: HTTP 400 `ValidationException` (empty string not allowed for key) ¹⁷.
- **Conditional update failure:** Provide a `ConditionExpression` that evaluates false (e.g. `ConditionExpression: "attribute_not_exists(id)"` on an existing item).
Expected: HTTP 400 `ConditionalCheckFailedException` with message "The conditional request failed." ²⁰. No changes made.

- **Idempotency of Update:** Applying the same update twice (that has no effect second time) should succeed both times.
Expected: HTTP 200 on both calls. For example, setting an attribute to its current value yields no error.

DeleteItem

- **Delete existing item (no ReturnValues):**
Input: `{ "TableName": "T1", "Key": { "id": { "S": "123" } } }`.
Expected: HTTP 200. Response `{}` (if no `ReturnValues`). The item is removed.
- **Delete existing with ALL_OLD:** Same as above but add `"ReturnValues": "ALL_OLD"`.
Expected: HTTP 200. Response `Attributes` contains the deleted item's attributes as they were before deletion ²¹.
- **Delete non-existent item:**
Expected: HTTP 200 (idempotent) with no error ²². No `Attributes` returned even if `ALL_OLD` requested. DynamoDB does not fail deleting a missing item.
- **Missing TableName or Key:**
Expected: HTTP 400 `ValidationException` ².
- **Missing sort key on composite table:** Only HASH given.
Expected: HTTP 400 `ValidationException` (missing key part).
- **Key type mismatch:** Wrong type for key attribute.
Expected: HTTP 400 `ValidationException` (type mismatch) ¹⁶.
- **Empty string or null key:** Invalid.
Expected: HTTP 400 `ValidationException` (empty key) ¹⁷.
- **Conditional delete failure:** Supply a `ConditionExpression` that is false.
Expected: HTTP 400 `ConditionalCheckFailedException` (DynamoDB states "The conditional request failed.") ²³. Item remains.
- **Idempotency of Delete:** Call `DeleteItem` twice on the same key.
Expected: HTTP 200 both times ²². The first call deletes the item, the second sees no item and still succeeds with no error.

Note: In many responses (e.g. `CreateTable` or `DescribeTable`), fields like timestamps, request IDs, or UUIDs will vary between runs. Tests should normalize or ignore these dynamic fields to allow deterministic comparisons. All HTTP status codes and error codes above refer to DynamoDB Local behavior, which db.shvm.in should match for core CRUD and table operations.

Sources: AWS DynamoDB API documentation and examples ¹⁸ ¹⁴ ¹³ ¹⁰, community examples ³ ⁴. These describe DynamoDB's expected behavior for missing parameters, type mismatches, error codes, and idempotency ² ²², which db.shvm.in should emulate.

¹ ⁵ `CreateTable` - Amazon DynamoDB

https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_CreateTable.html

² Error handling with DynamoDB - Amazon DynamoDB

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Programming.Errors.html>

3 **DynamoDB - Missing validation on Index-attributes** · Issue #2445 · getmoto/moto · GitHub

<https://github.com/getmoto/moto/issues/2445>

4 **python - Ceating dynamodb table says "invalid One or more parameter values were invalid: Some index key attributes are not defined in AttributeDefinitions" - Stack Overflow**

<https://stackoverflow.com/questions/62918389/ceating-dynamodb-table-says-invalid-one-or-more-parameter-values-were-invalid>

6 7 **ListTables - Amazon DynamoDB**

https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_ListTables.html

8 9 10 11 **DescribeTable - Amazon DynamoDB**

https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_DescribeTable.html

12 13 **DeleteTable - Amazon DynamoDB**

https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_DeleteTable.html

14 16 17 **PutItem - Amazon DynamoDB**

https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_PutItem.html

15 **amazon web services - How to resolve the issue of (ValidationException) in AWS DynamoDB while working with CLI? - Stack Overflow**

<https://stackoverflow.com/questions/59456809/how-to-resolve-the-issue-of-validationexception-in-aws-dynamodb-while-working>

18 **GetItem - Amazon DynamoDB**

https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_GetItem.html

19 20 **UpdateItem - Amazon DynamoDB**

https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_UpdateItem.html

21 22 23 **DeleteItem - Amazon DynamoDB**

https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_DeleteItem.html